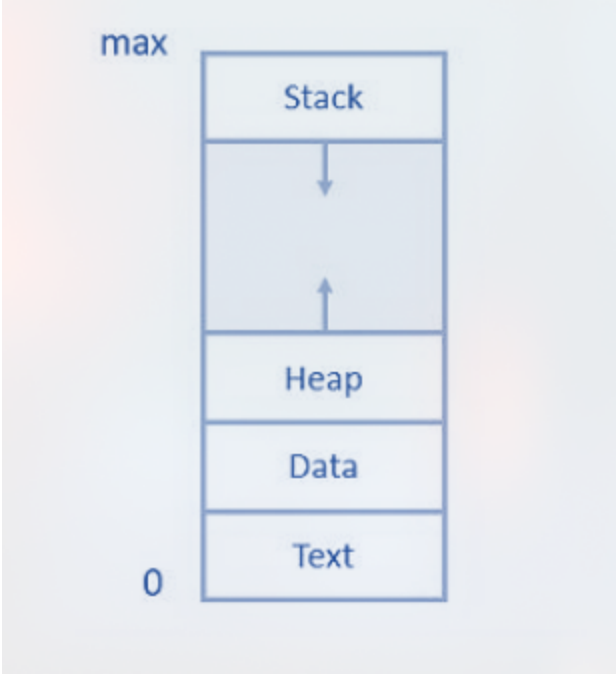
Lesson 3

What is a Process?

- Program in execution
- Has its **own address space**
- Has **stack, heap, registers**
- **Executes independently**



Why Do Processes Need to Communicate?

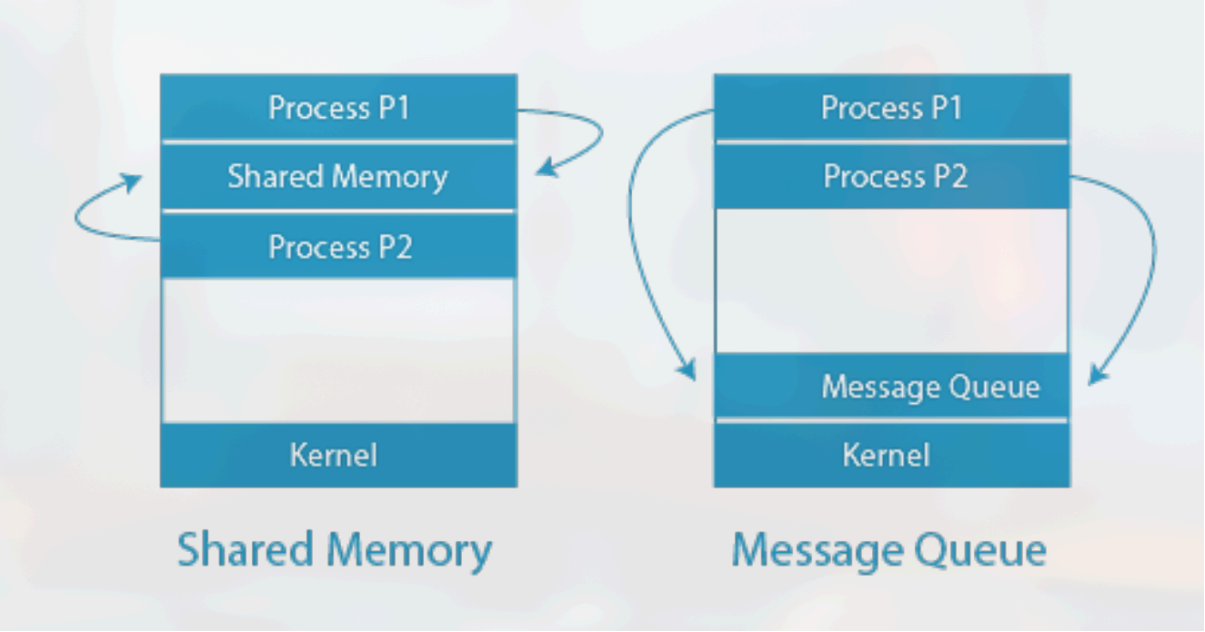
- Information sharing
- Computation speed-up
- Modularity
- Convenience

  Example:

- In a browser like Google Chrome:
  - One process handles UI
  - One handles rendering
  - One handles networking

Inter-Process Communication (IPC)

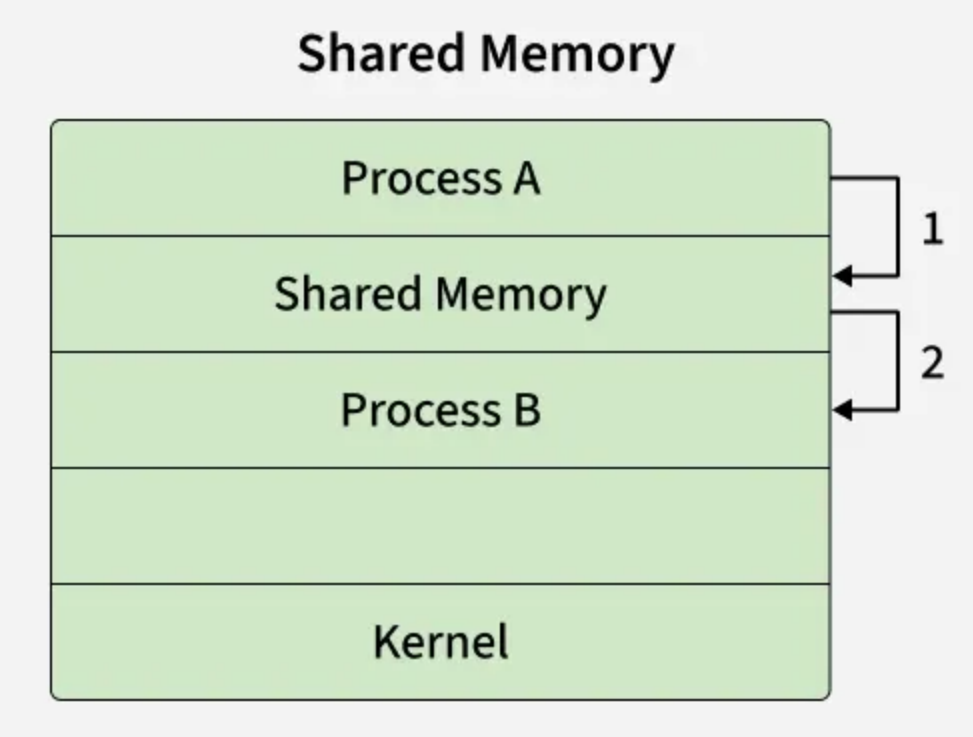
- Inter-Process Communication (IPC) is an operating system mechanism enabling independent processes to **exchange data, synchronize actions, avoid conflicts while accessing shared resources, and share information.**
- It ensures:
  - Safe communication
  - Data integrity
  - Controlled access
  - Conflict avoidance



Shared Memory Model

- OS creates a shared segment
- Processes attach
- Read/write like normal memory

- After setup, the kernel is not involved in every read/write.



- In the above shared memory model, a common memory space is allocated by the kernel.
- Process A writes data into the shared memory region (Step 1).
- Process B can then directly read this data from the same shared memory region (Step 2).
- Since both processes access the same memory segment, this method is fast but requires synchronization mechanisms (like semaphores) to avoid conflicts when multiple processes read/write simultaneously.
- Example: Multiple people can edit the document at the same time in shared google doc.

Advantages

- Very fast
- No message copying
- Efficient for large data

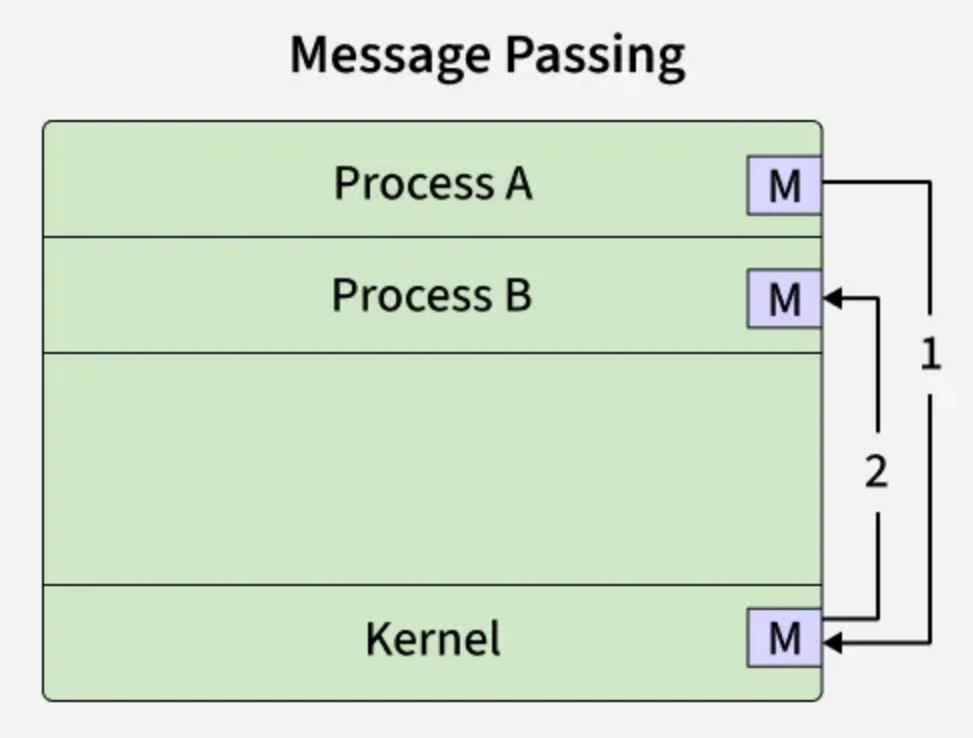
Problems

- No automatic synchronization

- Race conditions possible
- Synchronization errors

Message Passing

Message Passing is a method where processes communicate by sending and receiving messages to exchange data. One process sends a message and the other process receives it, allowing them to share information. **Message Passing can be achieved through different methods like Sockets, Message Queues or Pipes.**



- In the above message passing model, processes exchange information by sending and receiving messages through the kernel.
- Process A sends a message to the kernel (Step 1).
- The kernel then delivers the message to Process B (Step 2).
- Here, processes do not share memory directly. Instead, communication happens via system calls (send(), recv(), or similar).
- This method is simpler and safer than shared memory because there's no risk of overwriting shared data, but it incurs more overhead due to kernel involvement.

- Example: Multiple people send updates to a group chat, but each message goes through the server before others see it, like processes sending messages instead of directly sharing memory.

Message Queue

- Processes communicate by exchanging messages
- No shared address space
- OS kernel manages communication

Basic operations:

```
• send(message)
• receive(message)
```

Communication Types:

```
• Direct or Indirect
• Blocking (Synchronous) or Non-blocking (Asynchronous)
```

Direct

In Direct IPC, each process that wants to communicate must explicitly name the recipient or sender.

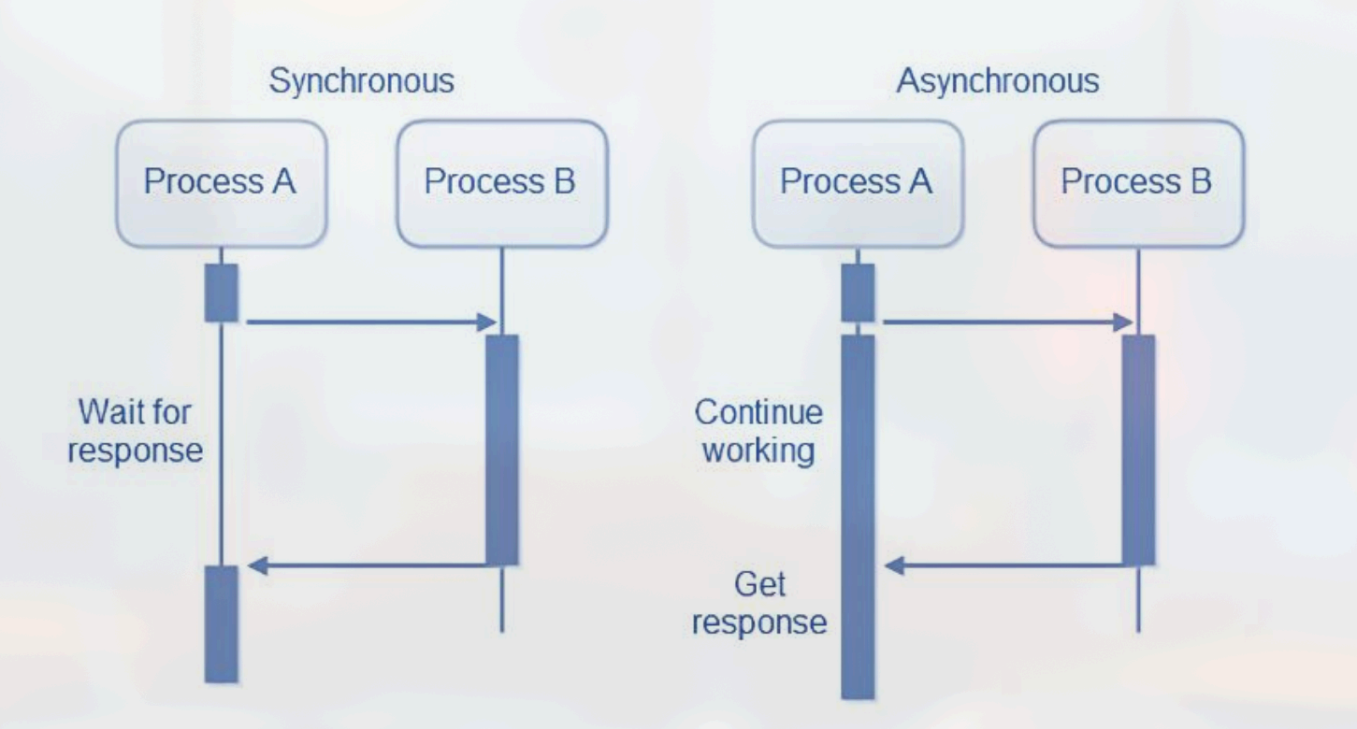
- Process A sends directly to Process B.
- Both must know each other.
  Ex: Sender and receiver are in the same LAN. The sender knows the exact IP and MAC addresses of the recipient and sends the packet using Layer 2 mechanisms without needing a router.

Indirect

In indirect IPC, processes do not know the each other exactly. They know about a shared location or **mailbox** that the other process can access.

- Uses mailbox/port.
- Processes are sent to the mailbox instead of being processed directly.
  Ex: The sender and receiver are in different LANs. The sender does not know the exact L2 addressing of the receiver. Therefore, the sender sends the packets to the default gateway(mailbox).

Blocking vs Non-Blocking Communication



Aspect	Blocking	Non-Blocking
Execution Flow	Halts program execution until completion	Allows program to continue with other tasks
Efficiency	May waste CPU time waiting for the current task	Utilizes CPU efficiently for other tasks.
Complexity	Simpler to implement and debug.	Requires additional constructs like callbacks.
Responsiveness	Can make programs unresponsive.	Maintains responsiveness of applications
Example	TCP	UDP

Sender and Receiver Block

Sender Block

- While a sender is sending a message to a receiver, the sender cannot send a message to another receiver process.

Receiver Block

- While a receiver process is receiving a message from one sender, it cannot receive a message from another sender until the first session is complete.